

CS1101S Cheatsheet

for midterms AY26-27
adapted from m. zaidan

Recursion & Iteration

Recursion: **Increasing** deferred operations

Iteration: **Constant** deferred operations

(rule of thumb: if the final call is the application call, it is usually *iterative*)

Recipe (wishful thinking): find the trivial base case; assume you can solve $n - 1$ (or $n/2$), use it to solve n . **5 steps:** read carefully → play with examples (incl. 0, negative) → think (existing solution? divide & conquer?) → program → test.

Substitution model (applicative order)

- **Primitive** expr → its value. **Operator combination** → evaluate operands, apply operator.
- **Declaration** $x = e \rightarrow$ evaluate e , replace x by the value in the rest.
- **Application** → evaluate function & arguments first; primitive: apply it; compound: substitute argument values for parameters in the return expression.
- a if p else $b \rightarrow$ evaluate p first, then **only** the chosen branch.
- **Normal order:** substitute unevaluated args, evaluate only when needed (Python does **not** do this).

```
def factorial(n): # recursive: Θ(n) time & space
    return 1 if n == 0 else n * factorial(n - 1)
# 4 * (3 * (2 * (1 * factorial(0)))) <- deferred ops

# fact_iter(1, 0, n): Θ(n) time, Θ(1) space
def fact_iter(product, counter, n):
    return (product if counter == n
            else fact_iter(product * (counter + 1),
                           counter + 1, n))
```

```
def fib(n): # tree rec.: Θ(φn) time, Θ(n) space
    return n if n <= 1 else fib(n - 1) + fib(n - 2)

def fib_iter(a, b, count): # fib_iter(1, 0, n): Θ(n)
    return b if count == 0
           else fib_iter(a + b, a, count - 1))
```

```
def repeat_apply(f, n, x): # recursive: f deferred
    return x if n == 0 else f(repeat_apply(f, n - 1, x))
def repeat_apply(f, n, x): # iterative: f done first
    return x if n == 0 else repeat_apply(f, n - 1, f(x))
```

```
def cc(amt, kinds): # ways to make change
    return 1 if amt == 0
           else 0 if amt < 0 or kinds == 0
           else cc(amt - first_denomination(kinds),
                  kinds)
           + cc(amt, kinds - 1))
```

```
def pascal(row, pos): # both counted from 0
    return 1 if pos == 0 or pos == row
           else pascal(row - 1, pos - 1)
           + pascal(row - 1, pos)
```

```
def gcd(a, b): # Euclid; iterative
    return a if b == 0 else gcd(b, a % b)
```

Evaluation, Syntax & Scope

Precedence	Form	Example
1 (highest)	function application	$f(x)$
2	conditional expression	x if p else y
3 (lowest)	lambda expression	$\lambda x. e$

The lambda body swallows the whole conditional:

```
lambda x: x if n == 0 else g(x)
# = lambda x: (x if n == 0 else g(x))
```

and/or short-circuit (syntactic forms, not operators):

```
p and q # = q if p else False
p or q # = True if p else q
False and boom(); True or boom() # boom never runs
```

Conditional statements: every branch must return, else the function returns None. A declaration inside a branch only runs if that branch runs.

Declarations: (1) primitive names (incl. from rune import heart), (2) declaration assignments – scope is the **whole body** of the immediately surrounding function (or program), (3) parameters – scope is the function body, (4) $\text{def } f - \text{as if } f = \dots$ **Rule:** a name refers to the **closest surrounding declaration** (lexical scope).

```
z = 2
def f(g):
    z = 4
    return g(z)
f(lambda y: y + z) # 6: lambda's z is global z=2

z = 3
def h():
    print(z) # ERROR: local z below shadows the
             # global z in all of h; no value yet
    z = 4
```

Same error if $z = \dots$ sits in an if branch that did not run.

Higher-Order Functions

Functions are values: pass them in, return them, store them.

```
def sum(term, a, next, b):
    return (0 if a > b
            else term(a) + sum(term, next(a), next, b))
sum(lambda x: x * x * x, 1, lambda x: x + 2, 9)
# = 13 + 33 + 53 + 73 + 93
```

```
compose = lambda f, g: lambda x: f(g(x))
def make_adder(x): return lambda y: x + y
make_adder(4)(6) # 10
```

```
def repeat(f, n): # returns a fn
    return lambda x: (x if n == 0
                      else repeat(f, n - 1)(f(x)))
repeat(lambda n: n + 1, 3)(4) # 7
```

Types: $\text{math_sqrt} : \text{Number} \rightarrow \text{Number}$ (Number-Transformation)

$\text{Curve} := \text{Number} \rightarrow \text{Point}$ (t in [0, 1]; $\text{make_point}, x_{\text{of}}, y_{\text{of}}$)

$\text{Wave} := \text{Number} \rightarrow \text{Number}$ (time → amplitude in [-1, 1]);

$\text{make_sound} : (\text{Wave}, \text{Number}) \rightarrow \text{Sound}$

Scott numerals (past MT):

```
zero = lambda z: lambda s: z
one = lambda z: lambda s: s(zero) # s gets pred.
add_one = lambda n: lambda z: s(n)
to_int = lambda n: n(0)(lambda p: 1 + to_int(p))
```

Definitions of List & Tree

```
pair = lambda x, y: (lambda f: f(x, y))
head = lambda p: p(lambda x, y: x)
tail = lambda p: p(lambda x, y: y)
# alt: pair = lambda x, y: lambda i: x if i == 0 else y
# head = lambda p: p(0); tail = lambda p: p(1)
```

A linked list is either **None** or a **pair whose tail is a linked list**.

A tree is either **None** or **head is an element, tail is a tree** or **head is a tree, tail is a tree** (*no trees of Nones or of pairs – they'd be mistaken for subtrees*)

Notations

$\text{print}(\text{pair}(1, 2)) \rightarrow [1, 2]$ (box notation)

$\text{print}(\text{l1ist}(1, 2, 3)) \rightarrow [1, [2, [3, \text{None}]]]$

$\text{print_l1ist}(\text{pair}(\text{pair}(7, 8), \text{l1ist}(1, 2)), 6) \rightarrow$
 $[[\text{l1ist}([7, 8], 1, 2), 6]]$ (l1ist notation)

Box-and-pointer: one box per $\text{pair}(\dots)$ call; $\text{None} = \text{slash}$. $\text{draw_data}(x)$ shows it. Only $\text{pair}(\dots)$ creates a pair; head, tail, naming or passing a pair never copies it.

Pre-declared (Python §2)

$\text{l1ist}(1, 2, 3), \text{is_pair}, \text{is_none}, \text{is_l1ist}, \text{length}, \text{append}, \text{reverse}, \text{remove}(\text{first}), \text{remove_all}, \text{l1ist_ref}(xs, n)$ (0-indexed, $O(n)$), $\text{enum_l1ist}(2, 5) = \text{l1ist}(2, 3, 4, 5)$, $\text{build_l1ist}(f, n) = \text{l1ist}(f(0), \dots, f(n-1))$, for_each , $\text{is_number}, \text{is_string}, \text{is_function}, \text{min}, \text{max}, \text{str.reduce}(\text{op}, \text{init}, \text{l1ist}(1, 2, 3)) = \text{op}(1, \text{op}(2, \text{op}(3, \text{init})))$ (right fold).

Source built-ins, in Python

```
def is_l1ist(xs):
    return (True if is_none(xs)
            else is_l1ist(tail(xs)) if is_pair(xs)
            else False)

def length(xs):
    return 0 if is_none(xs) else 1 + length(tail(xs))

def length_iter(xs):
    def helper(ys, n):
        return (n if is_none(ys)
                else helper(tail(ys), n + 1))
    return helper(xs, 0)

def append(xs, ys): # copies xs, shares ys: Θ(|xs|)
    return (ys if is_none(xs)
            else pair(head(xs), append(tail(xs), ys)))

def remove(x, xs): # first occurrence only
    return (None if is_none(xs)
            else tail(xs) if head(xs) == x
            else pair(head(xs), remove(x, tail(xs))))

def reverse(xs): # iterative; Θ(n) time and space
    def rev(orig, rev_so_far):
        return (rev_so_far if is_none(orig)
                else rev(tail(orig),
                        pair(head(orig), rev_so_far)))
    return rev(xs, None)
# append(reverse(tail(xs)), l1ist(head(xs))) : Θ(n2)
# pair(reverse(tail(xs)), head(xs)) : WRONG, nested
```

List Abstractions

```
def map(f, xs):
    return (None if is_none(xs)
            else pair(f(head(xs)), map(f, tail(xs))))
```

Recursive process; **time: $O(n)$, space: $O(n)$** , where n is the length of xs .

```
def filter(pred, xs):
    return (None if is_none(xs)
            else pair(head(xs), filter(pred, tail(xs)))
            if pred(head(xs))
            else filter(pred, tail(xs)))
```

Recursive process; **time: $O(n)$, space: $O(n)$** , where n is the length of xs .

```
def reduce(op, initial, xs):
    return (initial if is_none(xs)
            else op(head(xs),
                    reduce(op, initial, tail(xs))))
```

$\text{reduce}(\lambda \text{curr}, \text{wish}: \dots, \text{initial}, \text{xs})$

Recursive process; **time: $O(n)$, space: $O(n)$** , where n is the length of xs assuming op takes constant time.

```
my_map = lambda f, xs: reduce(
    lambda x, acc: pair(f(x), acc), None, xs)
flatten = lambda xss: reduce(append, None, xss)
copy = lambda xs: map(lambda x: x, xs) # new pairs
def remove_duplicates(xs):
    return (None if is_none(xs)
            else pair(head(xs), remove_duplicates(
                filter(lambda y: y != head(xs),
                       tail(xs)))))
```

Tree Abstractions

```
def reduce_tree(f, op, initial, tree):
    return reduce(lambda x, ys:
        op(reduce_tree(f, op, initial, x), ys)
        if is_l1ist(x)
        else op(f(x), ys),
        initial, tree)

def map_tree(f, tree):
    return map(lambda sub_tree:
        f(sub_tree) if not is_l1ist(sub_tree)
        else map_tree(f, sub_tree),
        tree)
```

```
def flatten_tree(xs):
    def h(xs, prev):
        return (prev if is_none(xs)
                else append(flatten_tree(xs), prev)
                if is_l1ist(xs)
                else pair(xs, prev))
    return reduce(h, None, xs)
```

```
def count_data_items(tree):
    return (0 if is_none(tree)
            else count_data_items(head(tree))
            + count_data_items(tail(tree))
            if is_l1ist(head(tree))
            else 1 + count_data_items(tail(tree)))

scale_tree = lambda t, k: map_tree(lambda x: x * k, t)
```

Binary Search Trees

Binary tree: empty, or (entry, left subtree, right subtree). BST: everything in left < entry < everything in right. find/insert: $O(\log n)$ if balanced, $O(n)$ worst case (degenerate, e.g. inserted in sorted order).

```
def make_empty_tree(): return None
def is_empty_tree(t): return is_none(t)
def make_tree(e, l, r): return l1ist(e, l, r)
def entry(t): return head(t)
def left_branch(t): return head(tail(t))
def right_branch(t): return head(tail(tail(t)))
```

```
def insert(bst, item):
    if is_empty_tree(bst):
        return make_tree(item, make_empty_tree(),
                          make_empty_tree())
    elif item < entry(bst):
        return make_tree(entry(bst),
                          insert(left_branch(bst), item),
                          right_branch(bst))
    elif item > entry(bst):
        return make_tree(entry(bst),
                          left_branch(bst),
                          insert(right_branch(bst), item))
    else:
        return bst
```

```
def find(bst, name):
    return (False if is_empty_tree(bst)
            else True if name == entry(bst)
            else find(left_branch(bst), name)
            if name < entry(bst)
            else find(right_branch(bst), name))
```

Permutations & Combinations

```
def permutations(s):
    return (l1ist(None) if is_none(s)
            else reduce(append, None,
                        map(lambda x: map(lambda p: pair(x, p),
                                           permutations(remove(x, s))),
                           s)))
```

```
def subsets(s):
    return reduce(
        lambda x, s1: append(s1,
                              map(lambda ss: pair(x, ss), s1)),
        l1ist(None),
        s)
```

```
def choose(n, r):
    if n < 0 or r < 0:
        return 0
    elif r == 0:
        return 1
    else:
        to_use = choose(n - 1, r - 1)
        not_to_use = choose(n - 1, r)
        return to_use + not_to_use
```

```
def combinations(xs, r):
    if (r != 0 and is_none(xs)) or r < 0:
        return None
    elif r == 0:
        return l1ist(None)
    else:
        no = combinations(tail(xs), r)
        yes = combinations(tail(xs), r - 1)
        yes_item = map(lambda x: pair(head(xs), x),
                       yes)
        return append(no, yes_item)

def makeup_amount(x, coins):
    if x == 0:
        return l1ist(None)
    elif x < 0 or is_none(coins):
        return None
    else:
        combi_A = makeup_amount(x, tail(coins))
        combi_B = makeup_amount(x - head(coins),
                                tail(coins))
        combi_C = map(lambda x: pair(head(coins), x),
                      combi_B)
        return append(combi_A, combi_C)
```

Orders of Growth

for $r(n)$,

Big O $O(g(n))$: if there is a positive constant k such that $r(n) \leq k \cdot g(n)$ for any sufficiently large value of n

Big Theta $\Theta(g(n))$: if there are positive constants k_1 and k_2 and a number n_0 such that $k_1 \cdot g(n) \leq r(n) \leq k_2 \cdot g(n)$ for any $n > n_0$.

Big Omega $\Omega(g(n))$: if there is a positive constant k such that $k \cdot g(n) \leq r(n)$ for any sufficiently large value of n

Order (ascending): $1, \log n, n, n \log n, n^2, n^3, 2^n, 3^n, n^n$

Ignore constants, lower order terms. $O(2n) = O(3n) = O(n)$

For a sum, take the larger term. $O(n) + O(n^2) = O(n^2)$

For a product, multiply the two terms. $O(n) \times O(n) = O(n^2)$

$\Theta \Rightarrow$ both O and Ω . O is only an upper bound: factorial is $O(n)$ and $O(n^2)$ – give the tightest. fib is $\Theta(\varphi^n)$, $\varphi = (1 + \sqrt{5})/2$, so also $O(2^n)$.

Space

Space = max of (deferred operations) + (pairs that must be remembered at the same time – not total created) + (function values/closures created, e.g. CPS). `pair(...)` creates; `head/tail/passing` doesn't. A pair made in a frame and not returned is garbage after the next call.

`append(xs, ys)`: new copy of `xs, ys` shared $\Rightarrow \Theta(n)$ time & space, n = length of `xs`. `map/filter` build new pairs.

Recurrence Relations

$T(n)$	Solution
$O(1) + T(n-1)$	
$O(1) + 2T(n/2)$	$O(n)$
$O(n) + T(n/2)$	
$O(1) + T(n/2)$	$O(\log n)$
$O(\log n) + T(n-1)$	$O(n \log n)$
$O(n) + 2T(n/2)$	
$O(n) + T(n-1)$	$O(n^2)$
$O(n^k) + T(n-1)$	$O(n^{k+1})$
$O(n) + 2T(n-1)$	$O(2^n)$
$O(1) + 2T(n-1)$	

Master Theorem

For $T(n) = a T(n/b) + f(n)$, with $a \geq 1, b > 1$:

a = no. of subproblems, n/b = size of each, $f(n)$ = work to split and combine.

Shortcut when $f(n) = \Theta(n^d)$: compare a with b^d .

$a < b^d \Rightarrow \Theta(n^d)$ root wins

$a = b^d \Rightarrow \Theta(n^d \log n)$ tie

$a > b^d \Rightarrow \Theta(n^{\log_b a})$ leaves win

Recurrence	a, b, d	Case	Result
Binary search			
$T(n/2) + O(1)$	1, 2, 0	tie	$\Theta(\log n)$
Merge sort			
$2T(n/2) + O(n)$	2, 2, 1	tie	$\Theta(n \log n)$
Tree traversal			
$2T(n/2) + O(1)$	2, 2, 0	leaves	$\Theta(n)$
$T(n/2) + O(n)$	1, 2, 1	root	$\Theta(n)$

Only for splitting by a factor (n/b). For $T(n-1)$ recurrences, use the table above.

Sorting/Search Algorithms

	Time		Space
	best	worst	
Binary Search	$\Theta(\log n)$	$O(\log n)$	$O(1)$
Selection Sort	$\Theta(n^2)$	$O(n^2)$	$O(n)$
Insertion Sort	$\Theta(n)$	$O(n^2)$	$O(n)$
Merge Sort	$\Theta(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$\Theta(n \log n)$	$O(n^2)$	$O(n^2)$

Sorting = produce a permutation in non-decreasing order using **comparisons only**.

Insertion best case: already sorted (each insert is $O(1)$).

Quicksort worst: already sorted (pivot = min/max). Helpers:

`merge` $\Theta(n)$ (two lists of n), `take/drop` $\Theta(n)$, `middle` $\Theta(1)$, `length` $\Theta(n)$.

Binary Search

Halve the search space each step: $n = 2^k$ reaches 1 after $k = \log_2 n$ steps. Needs a total order and $O(1)$ access to the middle (a BST gives this).

```
def guess(start, end):
    if start == end:
        return start
    mid = math.floor((start + end) / 2)
    check = check_guess(mid)
    if check == "correct": return mid
    elif check == "too low":
        return guess(mid + 1, end)
    else: return guess(start, mid - 1)
```

Insertion Sort

```
def insert(x, xs):
    return (l1ist(x) if is_none(xs)
           else pair(x, xs) if x <= head(xs)
           else pair(head(xs), insert(x, tail(xs))))
```

```
def insertion_sort(xs):
    return (xs if is_none(xs)
           else insert(head(xs),
                       insertion_sort(tail(xs))))
```

With `insert_cmp(x, xs, cmp)` (put x first iff `cmp(x, head(xs))`):
ascending `lambda x, y: x <= y`; descending `x >= y`; reverse
input `lambda x, y: False`; unchanged `lambda x, y: True`.

Selection Sort

```
def smallest(xs):
    return reduce(lambda x, y: x if x < y else y,
                  head(xs), tail(xs))

def selection_sort(xs):
    if is_none(xs):
        return xs
    else:
        x = smallest(xs)
        return pair(x, selection_sort(remove(x, xs)))
```

Merge Sort

```
def merge(xs, ys):
    if is_none(xs):
        return ys
    elif is_none(ys):
        return xs
    else:
        x = head(xs)
        y = head(ys)
        return (pair(x, merge(tail(xs), ys)) if x < y
                else pair(y, merge(xs, tail(ys))))
```

`middle = lambda n: n // 2` # `math.floor(n / 2)`

```
def take(xs, n):
    return (None if n == 0
           else pair(head(xs), take(tail(xs), n - 1)))
```

```
def drop(xs, n):
    return xs if n == 0 else drop(tail(xs), n - 1)
```

```
def merge_sort(xs):
    if is_none(xs) or is_none(tail(xs)):
        return xs
    else:
        m = middle(length(xs))
        return merge(merge_sort(take(xs, m)),
                     merge_sort(drop(xs, m)))
```

Quick Sort

```
def partition(xs, p):
    return pair(filter(lambda x: x <= p, xs),
                filter(lambda x: x > p, xs))

def quicksort(xs):
    if is_none(xs):
        return None
    elif is_none(tail(xs)):
        return xs
    else:
        pivot = head(xs)
        ptn = partition(tail(xs), pivot)
        return reduce(append, None,
                     l1ist(quicksort(head(ptn)), l1ist(pivot),
                           quicksort(tail(ptn))))
```

Continuation-Passing Style (CPS)

Passing the deferred operation as a function in an extra argument. Any recursive function can be converted this way.

```
def fac_cps(n, c):
    return (c(1) if n == 1
           else fac_cps(n - 1, lambda x: c(n * x)))

def append_cps(xs, ys, c):
    return (c(ys) if is_none(xs)
           else append_cps(tail(xs), ys,
                           lambda res: c(pair(head(xs), res))))

def reduce_cps(op, initial, xs, c):
    return (c(initial) if is_none(xs)
           else reduce_cps(op, initial, tail(xs),
                           lambda res: c(op(head(xs), res))))
```

Trace: `fac_cps(3, I) → fac_cps(2, lambda r: I(3*r)) → fac_cps(1, lambda r: I(3*(2*r))) → c(1) = 3*(2*1) = 6`
`I = lambda x: x`

Iterative process, but **space is $O(n)$** (the chain of continuation functions).

Also: iterative `append = append_iter(reverse(xs), ys)` where `append_iter` moves heads onto `ys` one at a time.

Data Abstraction

Constructors (`make_rat`), **selectors** (`numer`, `denom`), **predicates** (`is_sum`). Users see only these; **specification** (what) vs **implementation** (how).

```
def make_rat(n, d):
    g = gcd(n, d)
    return pair(n // g, d // g)

def numer(x): return head(x)
def denom(x): return tail(x)

def add_rat(x, y):
    return make_rat(numer(x) * denom(y)
                    + numer(y) * denom(x),
                    denom(x) * denom(y))

def mul_rat(x, y):
    return make_rat(numer(x) * numer(y),
                    denom(x) * denom(y))

def div_rat(x, y):
    return make_rat(numer(x) * denom(y),
                    denom(x) * numer(y))

def equal_rat(x, y):
    return numer(x) * denom(y) == numer(y) * denom(x)
```

Abstraction barriers: programs $\rightarrow$ `add_rat ...` $\rightarrow$ `make_rat`,
`numer`, `denom` $\rightarrow$ `pair`, `head`, `tail` $\rightarrow$ however pairs are implemented.

Symbolic Processing

Represent expressions as **tagged llists**; strings are variables.

```
def make_sum(a1, a2): return l1ist("+", a1, a2)
def make_product(m1, m2): return l1ist("*", m1, m2)
def addend(s): return head(tail(s)) # multiplier
def augend(s): return head(tail(tail(s))) # multiplic.
def is_sum(x): return is_pair(x) and head(x) == "+"
def is_product(x): return is_pair(x) and head(x) == "*"
def is_variable(x): return is_string(x)
def is_same_variable(v1, v2):
    return (is_variable(v1) and is_variable(v2)
            and v1 == v2)
```

```
def deriv_symbolic(exp, x):
    if is_number(exp):
        return 0
    elif is_variable(exp):
        return 1 if is_same_variable(exp, x) else 0
    elif is_sum(exp):
        return make_sum(
            deriv_symbolic(addend(exp), x),
            deriv_symbolic(augend(exp), x))
    elif is_product(exp):
        return make_sum(
            make_product(
                multiplier(exp),
                deriv_symbolic(multiplicand(exp), x)),
            make_product(
                deriv_symbolic(multiplier(exp), x),
                multiplicand(exp)))
    else:
        return error(exp, "unknown expression type")
```

`eval_symbolic(exp, name, val)`: number $\rightarrow$ itself; variable $\rightarrow$ `val` if same name (else `math_nan`); sum $\rightarrow$ + of evaluated parts; product $\rightarrow$ *. **Simplifying constructors:** `make_sum: 0 + a  $\Rightarrow$  a`, both numbers $\Rightarrow$ `add, x + x  $\Rightarrow$  make_product(2, x)`.
`make_product: any 0  $\Rightarrow$  0, 1 * m  $\Rightarrow$  m`, both numbers $\Rightarrow$ multiply. **Numeric:** `deriv = lambda f: lambda x: (f(x + dx) - f(x)) / dx`.

Runes, Curves & Sound

```
def stackn(n, rune):
    return (rune if n == 1
           else stack_frac(1 / n, rune,
                           stackn(n - 1, rune)))

def beside(r1, r2):
    return quarter_turn_left(
        stack(quarter_turn_right(r1),
              quarter_turn_right(r2)))

unit_circle = lambda t: make_point(
    math.cos(2 * math.pi * t),
    math.sin(2 * math.pi * t))

def connect_rigidly(c1, c2):
    return lambda t: (c1(2 * t) if t < 1 / 2
                     else c2(2 * t - 1))

draw_connected(200)(unit_circle)
wave = lambda t: math.sin(2 * math.pi * 440 * t)
a4 = make_sound(wave, 1.5) # (Wave, seconds)
```

Rule of Thumb for Abstractions

Is input a llist?	if not, don't use
Is <code>length(output) = length(input)</code> ?	use <i>map</i>
Are items in output = items in input?	use <i>filter</i>
Else	use <i>reduce</i>

Useful Math Functions

```
math_pow(base, exponent) # or base ** exponent
round(x) # banker's rounding: round(2.5) == 2
math_floor(x) # a // b is floor division
math_ceil(x); math_sqrt(x); abs(x); math_pi; math_e
a % b # remainder
```